Oshinei

Ref. OSH-CRM-2026-002
27 October 2026

To	Buzzebees Co., Ltd. — CRM PLUS project team
Cc	Easy POS — account manager and HO_CRM technical team
From	Oshinei — project owner, CRM integration
Re	Your document "API CRM PLUS_full - 2026.xlsx" (10 sheets, specification versions 1–3, 2022–2025)

Response to the CRM PLUS POS API proposal and requests required before development

Thank you for sending the CRM PLUS POS API specification. We have reviewed every sheet against our requirements and against live data from our Easy POS head-office database (HQ_Oshinei, read-only access, January–October 2026). This letter sets out our position, the evidence behind it, and the changes we need before development starts.

1. Summary

Position. Oshinei accepts the CRM PLUS POS API as the baseline for the integration. Member lookup (`/pos/profile`), point earning (`/pos/earn`) and coupon check/redeem (`/wallet/pay`) cover the core checkout flow.

Conditions. Four gaps would cause lost or duplicated customer value in normal operation and must be closed before development (requests R1–R4). Six further items are required before the pilot (R5–R10).

Change of roles. Your document describes a standard API that the POS must call. This means the POS-side development falls to Easy POS, not to Buzzebees as previously discussed. We ask Easy POS to confirm scope, effort and cost (section 7).

Ref.	Request to Buzzebees	Priority
R1	Add a unique reference (`refId`) and the receipt number to the earn APIs, and reject duplicates	Blocking
R2	Allow point reversal (`/earn/cancel`) after the day of sale, at least 30 days	Blocking
R3	Provide an API to reverse a redeemed coupon	Blocking
R4	Provide a minimal profile response for the POS (data minimisation under the PDPA)	Blocking
R5	Document the transaction status API (`/api/wallet/query`) and extend it to earn	Before pilot
R6	Provide a daily reconciliation report by branch, date and receipt	Before pilot
R7	Confirm which API burns points as a bill discount	Before pilot
R8	Confirm support for earning through a QR code printed on the receipt	Before pilot
R9	Issue Oshinei-specific credentials per branch and confirm production URLs	Before pilot
R10	Correct the inconsistencies in the specification (Appendix A)	Before development

We request a written reply to R1–R10 within 10 business days (proposed), followed by a joint meeting of Oshinei, Easy POS and Buzzebees.

Contents

1 Summary 1

2 Our current POS data (evidence from MSSQL) 4

3 Proposed integration architecture 7

4 Requirement coverage by the CRM PLUS API 8

5 Checkout sequence on the CRM PLUS API 9

6 Requests to Buzzebees (R1–R10) 10

7 Requests to Easy POS (E1–E5) 13

8 Items Oshinei will confirm (D1–D5) 14

9 Credential handling 15

10 Next steps 16

A Appendix A — Specification review, sheet by sheet 17

B Appendix B — Data sources and method 18

2. Our current POS data (evidence from MSSQL)

The figures below come from the Easy POS head-office database. They set the volume Buzzebees must support, show that no loyalty data exists today, and explain why several of our requests are necessary.

2.1 Volume and estate

500,905
finalised bills, Jan–Sep 2026

≈55,700
bills per month on average (47,754–69,475)

33
trading branches in Sep 2026 (27 in Jan)

33–35
POS terminals in use, about one per branch

Month 2026	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep
Finalised bills	69,475	63,003	50,381	54,507	61,337	47,754	50,806	54,314	49,328
Trading branches	27	27	27	29	30	31	32	33	33
Voided bills	295	233	263	254	258	275	251	317	337

2.2 Load profile

Payments concentrate in two peaks: lunch (13:00–14:00) and dinner (19:00–21:00). The busiest single hour in July–September 2026 closed **452 bills chain-wide** (12 August 2026, 13:00–14:00). The busiest single minute closed **15 bills**.

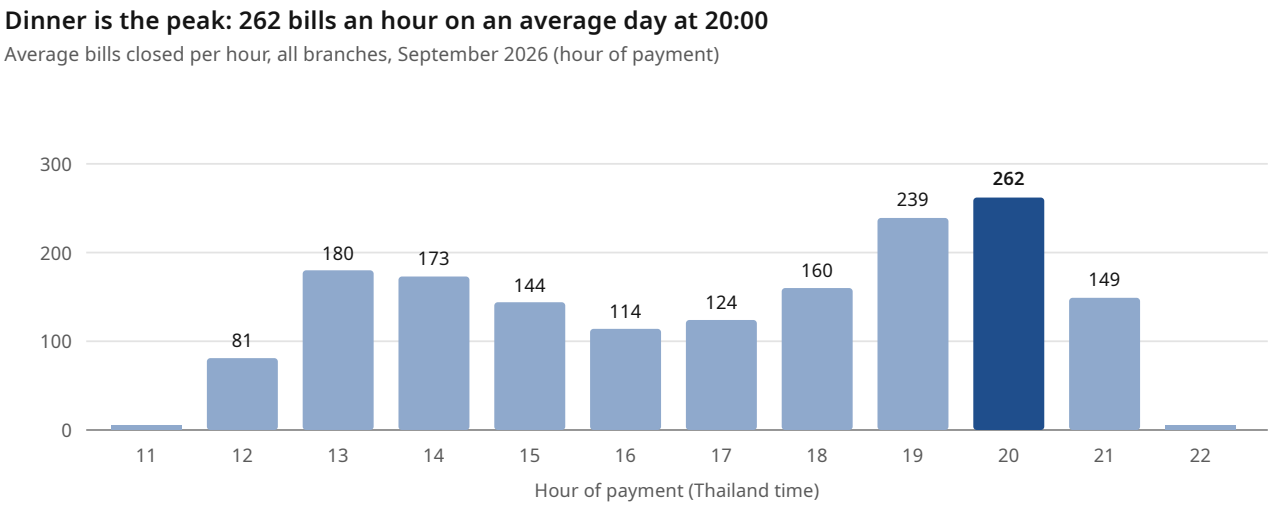


Figure 1. Average hourly bill volume, September 2026. Each paid bill triggers up to five API calls (section 5).

Implication for capacity. With up to five calls per bill, the busiest hour observed (452 bills) generates up to about 2,300 requests in one hour, and the busiest minute (15 bills) up to 75 requests. We ask Buzzebees to confirm the rate limit applied to an Oshinei token and that it covers at least 10 requests per second with headroom for branch growth.

2.3 No loyalty data exists today

Item in HQ_Oshinei	Value	Meaning
Bills with points earned or redeemed (`Sale.EarnPoint`, `Sale.RedeemedPoint`)	0 of 500,905 (Jan–Sep); 0 in Oct 1–26	No point scheme has ever run on this POS
Bills linked to a customer (`Sale.CustomerId`)	30,481 (6.1%), only 5–15 distinct customers a month	House and corporate accounts, not members
`Customer` table	7,128 rows; 1,938 phones, 53 e-mails, 18 birthdays	Tax-invoice records; not usable as a member base
Loyalty tables (`MemberCard`, `CustPoint`, `CustPointBalance`, `CustomerLoyalty`, `LoyaltyRank`, `LoyaltyPointDetail`, `MemberPrivilege`)	0 rows each	Easy's built-in membership module is unused

Consequence: the member base starts empty and every member will register through the Buzzebees LINE Official Account. There is no existing data to migrate.

2.4 Integration hooks already present in Easy POS

Object	Current state	Use in this integration
`ThirdParty` = `HO_CRM` (IsCRM = 1)	Active, last updated 21 Aug 2026	The connector through which the POS calls an external CRM
`PromotionByCRM`	0 rows; has CouponNumber, DiscountAmount, TransactionRef, IsSuccess, DiscountStatus, CouponSeriesCode	Where Easy records each coupon used and the Buzzebees transaction id
`CallAPILog`	0 rows; logs URL, method, body, response, status, response time, branch	Audit trail of every call to Buzzebees
`AssignThirdPartyToBranch` = `QrDataReceipt`	Assigned to 35 branches, all inactive	Receipt QR for self-service earning (R8)
`QRDataReceiptSetting`	0 rows (has QRPattern, public and secret key fields)	Receipt QR format to be agreed with Buzzebees
`ApiKey` for system "Easy"	Expired 8 Jul 2026	Easy to confirm whether it is needed (E4)

2.5 How a bill is identified

In September 2026 the 49,665 finalised bills had 49,665 unique branch + `SaleNum` pairs, but only 33,542 distinct `SaleId` values. `SaleId` therefore repeats across branches and cannot identify a bill on its own. `SaleNum` embeds the business date and a daily sequence, and return documents carry an `RE` prefix.

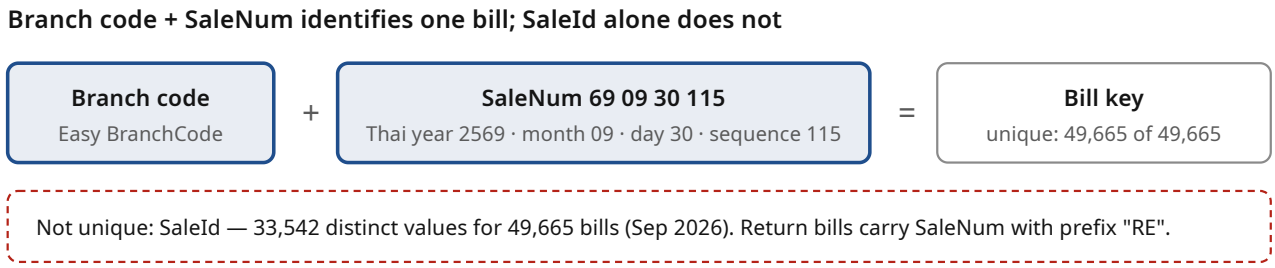


Figure 2. Bill identifier in Easy POS. We propose that the bill key is sent in every earn and redeem call (R1).

2.6 When bills are voided

Between January and September 2026, 2,482 finalised bills were voided. 2,446 (98.5%) were voided on the day of sale, but **36 were voided on a later day, up to 19 days after the sale**. Under the current rule that points can be reversed only within the day (sheet "Refund point"), every one of these 36 bills would leave points on a member account for a sale that no longer exists.

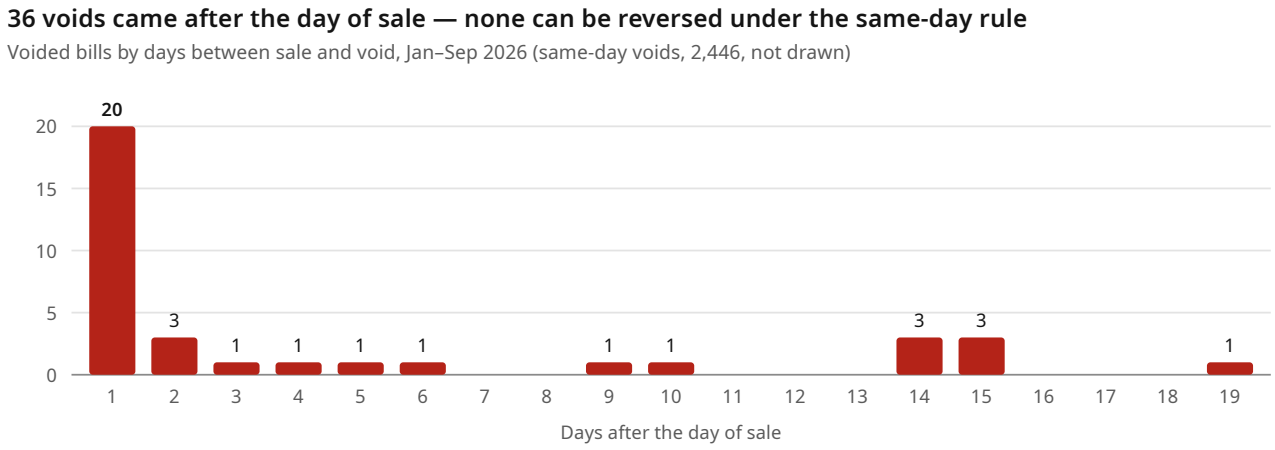


Figure 3. Voids made after the day of sale, by delay in days. Total 36; the longest delay was 19 days.

No return documents (`IsReturn`) were recorded in the period, so all reversals today are voids. If points are introduced, partial returns may appear; the cancel API must then support reversing a bill and earning again on the reduced amount (R2).

3. Proposed integration architecture

All calls originate from Oshinei systems and go to Buzzebees. POS terminals sit behind branch networks and cannot accept inbound calls. Solid lines are endpoints in your specification; dashed lines are items we request.

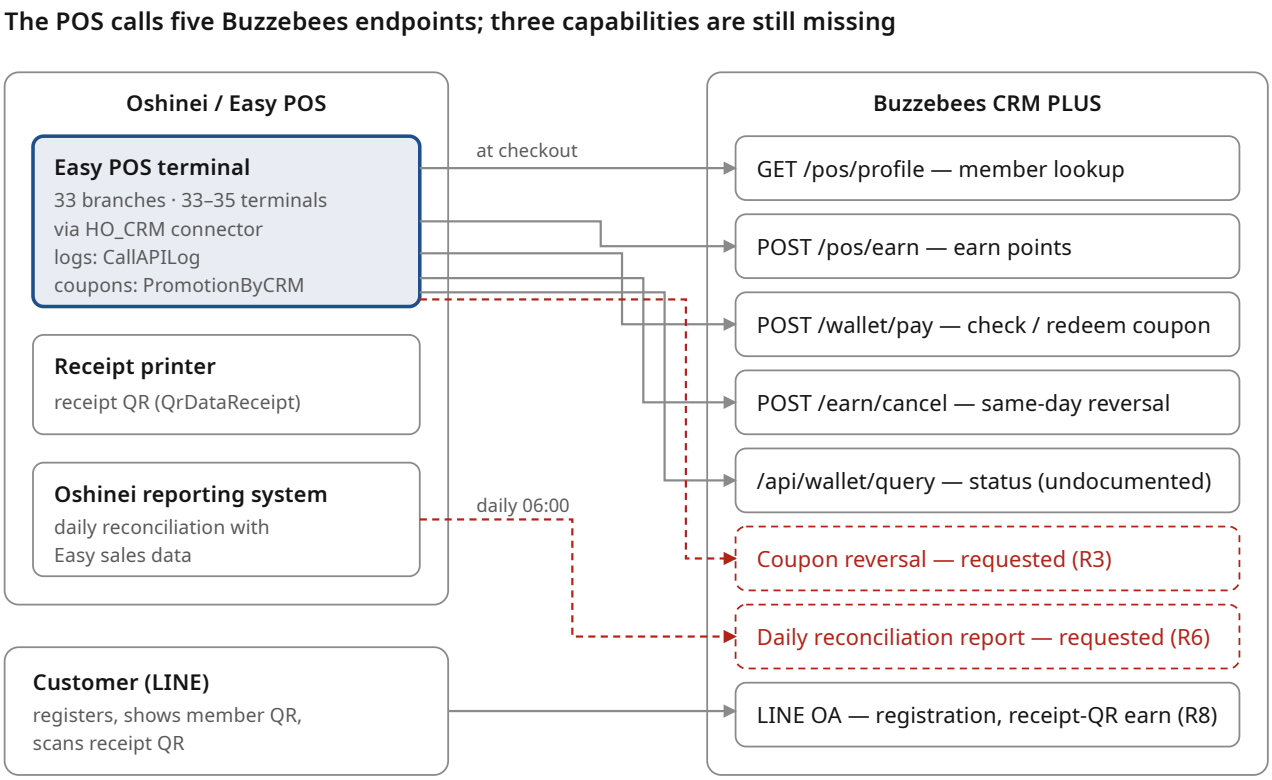


Figure 4. Proposed architecture. Red dashed items are not in the specification received and are requested in section 6.

Token model in your specification: one merchant token for all branches, with `branchId`, `brandId` and `terminalId` sent in each call. We request per-branch credentials (R9) so that a compromised branch can be revoked without stopping the chain.

4. Requirement coverage by the CRM PLUS API

Our specification (API Specification Draft 1.1, B0–B9) mapped to the endpoints in your document.

Ref.	Our requirement	CRM PLUS endpoint	Coverage	Gap
B0	Health check	—	Missing	POS cannot show "membership system unavailable" before a call fails. Acceptable if R5 exists; otherwise requested.
B1	Member lookup	`GET /pos/profile` by `code` or `contactNumber`	Partial	Returns full personal profile; no coupon list, point value or burn limits (R4).
B2	Validate coupon	`POST /wallet/pay` with `to_use=false`	Covered	Confirm returned discount type and value for bill- and item-level coupons.
B3	Redeem coupon	`POST /wallet/pay` with `amount=0`, `refId`	Covered	No reversal (R3).
B4	Burn points as discount	`POST /wallet/pay` payment mode?	Unclear	Payment mode appears to debit a wallet balance, not points (R7).
B5	Earn points	`/pos/earn`, `/pos/earn_direct`, `/pos/earn_direct_spending`	Partial	No unique reference or receipt number; retries can double-earn (R1).
B6	Reverse transactions	`POST /earn/cancel`	Partial	Same-day only; points only, not coupons (R2, R3).
B7	Master data	— (set up in Buzzebees back office)	Accepted	Buzzebees to issue `branchId` / `terminalId` per branch (R9).
B8	Transaction status	`/api/wallet/query` (mentioned, not specified)	Partial	No specification; no status for earn (R5).
B9	Reconciliation report	—	Missing	R6.
QR	Receipt-QR earning	—	Missing	R8.

Of eleven requirements, two are covered, five partly covered, one unclear, one accepted as an alternative and two missing. The four blocking gaps (R1–R4) all sit on the earn, reversal and lookup paths used on every bill.

5. Checkout sequence on the CRM PLUS API

How one bill would run on your endpoints. The POS calls at most five times; coupon redemption must succeed before the bill closes, and earning happens after payment so the receipt can print the points.

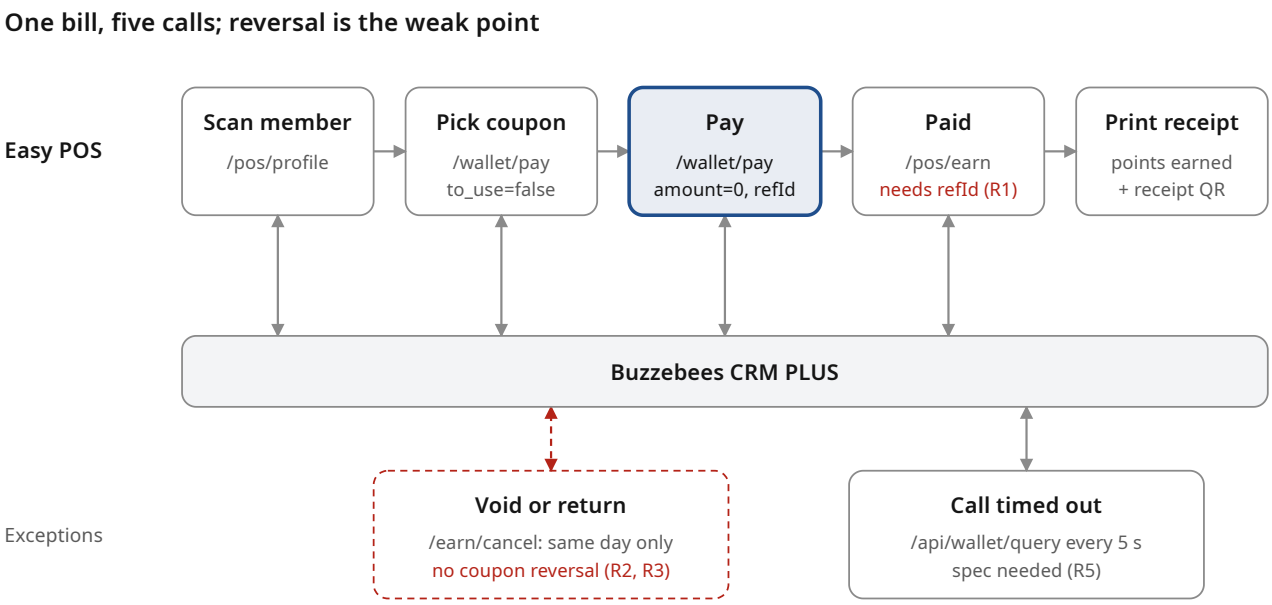


Figure 5. Checkout sequence mapped to CRM PLUS endpoints. Red marks show where the current API leaves customer value unprotected.

1. The cashier scans the member QR or enters a phone number; the POS calls `/pos/profile` and shows the masked name, point balance and usable coupons.
2. The customer picks a coupon; the POS calls `/wallet/pay` with `to_use=false` to check it without using it.
3. On payment the POS calls `/wallet/pay` with `amount=0` and a unique `refId` to redeem the coupon, and records the result in `PromotionByCRM`.
4. After payment the POS calls `/pos/earn` with the paid amount and the bill key, then prints the points earned on the receipt.
5. If a call times out, the POS queries the status before deciding; if the bill is later voided, the POS reverses both the earn and the coupon.

6. Requests to Buzzebees (R1–R10)

Each request states the current behaviour in your specification, the problem it causes, our evidence, and the change we ask for.

R1 — Unique reference and receipt number on earn Blocking

Current	Sheets "Earn", "Earn via user id" and "Earn Direct point": fields are code / contactNumber or userId, terminalId, amount, remark, remark2 (or description). There is no unique request reference and no receipt number. `/wallet/pay` already has `refId`.
Problem	When a call times out after Buzzebees has credited the points, the POS cannot tell success from failure. A retry credits the points a second time.
Evidence	Up to 452 bills an hour in peak trading (section 2.2), mostly on branch internet links. Even a small timeout rate produces duplicate credits every week.
Request	Add `refId` (unique per request, generated by the POS) and `receiptNo` (branch code + SaleNum, section 2.5) to all earn endpoints. A repeated `refId` must return the original result without crediting again; a repeated `receiptNo` must be rejected with a specific error id.

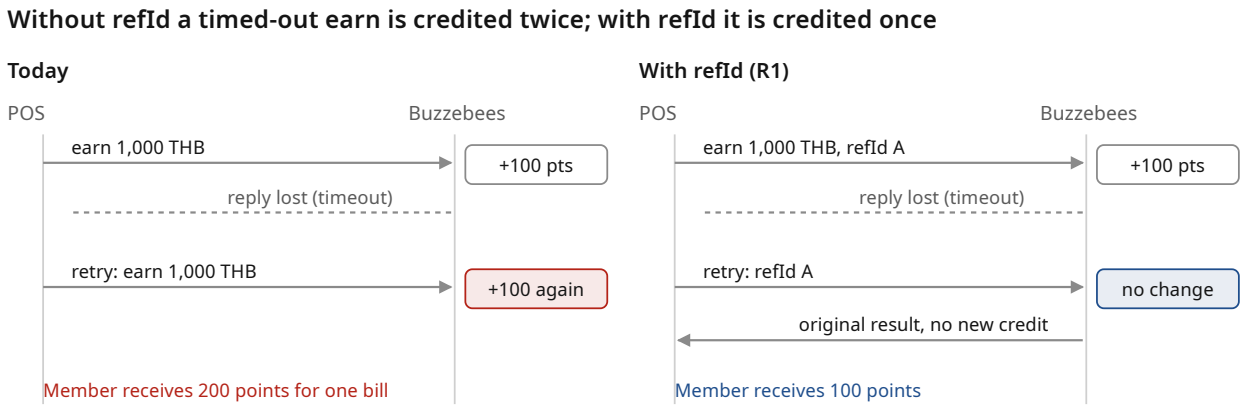


Figure 6. Duplicate credit on retry, and the effect of an idempotency reference.

R2 — Point reversal after the day of sale Blocking

Current	Sheet "Refund point": `/earn/cancel` with transactionId and terminalId; "Able to refund stamp only transaction within the day."
Problem	Bills voided after the day of sale keep their points; partial returns cannot be corrected.
Evidence	36 of 2,482 voids in Jan–Sep 2026 happened on a later day, up to 19 days after the sale (section 2.6, Figure 3).
Request	Allow reversal for at least 30 days after the sale. Accept `refId` or `receiptNo` as well as `transactionId`. If the member has already spent the points, define whether the balance may go negative or the shortfall is deducted from the next earn (Oshinei to decide, D4). A repeated cancel must return "already reversed" (your error 4027) without side effects.

R3 — Reversal of a redeemed coupon Blocking

Current	Sheets "Redeem" and "Check code": `/wallet/pay` redeems a code; there is no endpoint to reverse a redemption.
Problem	If a bill is cancelled after the coupon is used, for example when payment fails or the bill is voided, the customer loses the coupon although no discount was given.
Evidence	Voids run at about 250–340 bills a month (section 2.1). Each void of a coupon bill would need a manual correction in your back office.
Request	An endpoint that reverses a redemption by `transactionId` or `refId`, restores the code to unused, and is idempotent. The same 30-day window as R2.

R4 — Minimal profile for the POS Blocking

Current	Sheet "Profile": `/pos/profile` returns the full member record, including name, gender, birth date, age, contact number, e-mail, addresses, NationalIdCard, Passport, DriverLicenseNumber, credit-card fields, marketing consents and income.
Problem	A cashier screen needs only enough to confirm the customer and apply benefits. Receiving the rest on 33–35 shop terminals conflicts with data minimisation under the Personal Data Protection Act B.E. 2562, and Easy would store it in `CallAPILog`.
Request	A POS profile (or a field filter) that returns only: userId, masked display name (e.g. "Som***"), member level, point balance, points expiring within 30 days, and the list of coupons usable at the calling branch. No identity documents, address, birth date or payment data.

R5 — Transaction status API Before pilot

Current	Sheet "Redeem": status "unknown" requires calling `{url}/api/wallet/query` every 5 seconds, but the endpoint is not specified.
Request	Full specification (URL, parameters, response, terminal states) and coverage of earn and cancel as well as pay. The POS will query by `refId`. Please state the maximum time a transaction can stay "unknown".

R6 — Daily reconciliation report Before pilot

Current	No report or export is specified.
Request	A daily file or API, available by 06:00 for the previous day, one row per transaction: transactionId, refId, type (earn, cancel, redeem, coupon reversal), branchId, terminalId, receiptNo, userId, amount, points, coupon code, status, timestamp. Oshinei's accounting team will match it to Easy sales every day; mismatches to be corrected within 3 business days.

R7 — Burning points as a bill discount Before pilot

Current	`/wallet/pay` with `amount` greater than 0 is described as "payment" and returns a `balance`. It is not clear whether this debits points or a stored-value wallet.
Request	Confirm the endpoint and parameters for using points as a discount, the conversion rate, minimum and maximum per bill, and how the POS learns these values (ideally in the R4 profile).

R8 — Earning through the receipt QR Before pilot

Current	Not covered. Easy POS already has a receipt-QR feature assigned to 35 branches (inactive) with fields for a pattern and public/secret keys (section 2.4).
Request	Confirm whether the LINE Official Account can read a signed QR from the receipt (branch code, SaleNum, date, amount, signature) and credit points once per bill key. If yes, agree the format with Easy; if not, members must be identified at the counter on every bill.

R9 — Credentials and environments Before pilot

Current	One merchant token per merchant; production URL "TBC" in every sheet; sheet "url" lists new staging and production hosts from June 2024.
Request	Issue Oshinei UAT and production credentials, one per branch (or one token plus server-side branch binding); confirm the production base URLs; confirm the `branchId`, `brandId` and `terminalId` values for each Oshinei branch and how a new branch is added; state the token lifetime and rotation procedure (your error 1905 "token expired").

R10 — Specification corrections Before development

Please correct the items listed in Appendix A and send a single updated version of the document with a change log.

7. Requests to Easy POS (E1–E5)

Because CRM PLUS is a standard API that the POS must call, the POS-side work falls to Easy POS. We ask Easy to confirm the following.

Ref.	Request	Why
E1	Confirm that Easy will build the POS side through HO_CRM against the CRM PLUS API, with effort, cost and delivery date	Change of roles described in section 1
E2	Confirm HO_CRM supports the CRM PLUS call format: `Authorization: token` header, `application/x-www-form-urlencoded` bodies, and per-call `branchId` / `terminalId`	Different from the JSON format in our draft
E3	Confirm where results are written: member id (`Sale.CustomerId` or `CustRefNum`), points (`EarnPoint` , `RedeemedPoint` , `BalancePoint`), coupons (`PromotionByCRM`), and that every call is logged in `CallAPILog` with phone numbers masked	All are empty today (section 2.3–2.4)
E4	Confirm timeout handling: the bill always closes; earn is queued and retried with the same `refId` ; status is checked before any retry; queued items older than 24 hours are reported to Oshinei	Protects sales when Buzzebees is unreachable
E5	Confirm whether the expired `ApiKey` for system "Easy" (8 Jul 2026) is needed, and the receipt-QR format supported by `QRDataReceiptSetting`	Inputs to R8 and R9

8. Items Oshinei will confirm (D1–D5)

For transparency, these business rules are decided by Oshinei and will be sent separately. They do not block your reply to R1–R10.

Ref.	Decision	Oshinei's current preference
D1	Acceptance of Easy POS as the POS-side developer	Subject to E1
D2	Earn method: Buzzebees calculates points from the amount (`/pos/earn`) or the POS sends points (`/pos/earn_direct_spending`)	`/pos/earn` , so earn rules are maintained in one place
D3	Earn base: amount after discount, with or without VAT and service charge; whether Grab and LINE MAN bills earn	To be confirmed
D4	Reversal after points are spent: negative balance or deduction from the next earn	To be confirmed
D5	Pilot branches and start dates	Pilot in one branch first

9. Credential handling

> The document we received contains merchant tokens in clear text in several sheets, including tokens labelled "TMN only" and "All Except TMN". We have not copied these tokens into any other document and have restricted circulation of the file.
>
> We ask Buzzebees to (1) confirm whether these tokens are valid in any environment and revoke them if so, (2) remove tokens from specification documents in future, and (3) deliver Oshinei credentials through a separate secure channel (R9).

On our side, credentials will be stored only in the POS configuration managed by Easy and in Oshinei's secret store, never in documents, URLs or logs.

10. Next steps

No.	Step	Owner	Timing (proposed)
1	Written reply to R1–R10, with an updated specification	Buzzebees	Within 10 business days
2	Written reply to E1–E5, with effort and cost	Easy POS	Within 10 business days
3	Joint meeting to agree scope and the final API version	Oshinei (convenor)	After steps 1–2
4	UAT credentials and sandbox access for Easy and Oshinei	Buzzebees	After the meeting
5	Development and UAT against the test cases in our specification	Easy POS with Buzzebees	To be agreed
6	Pilot in one branch, daily reconciliation, go/no-go	Oshinei	To be agreed

Yours faithfully,

For Oshinei	Acknowledged by Buzzebees
Name:	Name:
Position: Project owner, CRM integration	Position:
Date:	Date:

Appendix A — Specification review, sheet by sheet

Observations on "API CRM PLUS_full - 2026.xlsx". Items marked ✱ need correction (R10).

Sheet	Content	Observation
Version	v1 2022-12-12 create; v2 2024-01-04 earn via user id; v3 2025-01-05 earn direct with spending	Each sheet still shows "Last Update Date 2022-12-12". Please align sheet dates with the version log. ✱
Profile	`GET /pos/profile`; code or contactNumber	Declared HTTP verb is GET with "Body" parameters and Content-Type form-urlencoded; the curl passes `contactNumber` as a query string. Please state query parameters for GET. ✱ Both code and contactNumber are marked mandatory although only one is sent. ✱ Full personal profile returned (R4).
Earn	`POST /pos/earn`; code/contactNumber, terminalId, amount, remark, remark2	No refId or receipt number (R1). `amount` is a string: please state decimal format and whether it is VAT-inclusive. ✱ Response "points" is null while "points_earn" carries the value; please confirm which field to use. ✱
Earn via user id	`POST /pos/earn_direct`; userId, terminalId, amount, description, remark	Field table has no `points`, but the curl sends `points="121.1"` and the error sample says "Parameter Missing or Invalid (points)". ✱
Earn Direct point	`POST /pos/earn_direct_spending`; userId, terminalId, amount, points, description, remark	The curl calls `/pos/earn_direct`, not `/earn_direct_spending`. ✱ Please state whether points may be decimal.
Refund point	`POST /earn/cancel`; token, transactionId, terminalId	Same-day limit (R2). Sample host is the old `stgwallet` domain, contrary to sheet "url". ✱
Redeem	`POST /wallet/pay`; code, refId, terminalId, branchId, brandId, amount, campaignId, stampcount	Status "unknown" refers to an unspecified query API (R5). Payment versus points not clear (R7). Response sample contains a national ID ("idcard"); please confirm this is not returned to the POS (R4). ✱
Check code	`POST /wallet/pay` with `to_use=false`, `transaction_id`	Uses `transaction_id` while Redeem uses `refId`; please confirm which identifier links check and redeem. ✱
Error Code	HTTP codes 400–500; custom ids 1414, 1905, 2048, 2526; payment 404, 4010, 10002	Please add ids for duplicate refId, duplicate receipt, reversal window expired and already reversed (4027 appears only in "Refund point"). ✱
url	Old and new staging/production hosts, change due 30 Jun 2024	Please confirm the new hosts are final and replace old hosts in all samples. ✱

Appendix B — Data sources and method

Item	Detail
Source	Easy POS head-office database HQ_Oshinei (Microsoft SQL Server), read-only account, queried 14 and 27 October 2026
Period	1 January – 30 September 2026 unless stated; points checked to 26 October 2026
Finalised bill	`Sale.IsFinalize = 1 AND Sale.Active = 1 AND ISNULL(Sale.IsReturn,0) = 0`
Voided bill	`Sale.IsFinalize = 1 AND Sale.Active = 0`; void delay = calendar days between `SaleDate` and `DeleteDate`
Hour of payment	Hour of `Sale.SaleDate` (bill closing time), Thailand time
Bill identifier test	Distinct (BranchId, SaleNum) and distinct SaleId over all finalised bills in September 2026
Loyalty usage	Bills with non-zero `EarnPoint` or `RedeemedPoint`; row counts of loyalty tables
Limitations	Branches not mapped in our reporting are included here (database level). Figures describe current operations; volumes after a loyalty launch may change.

Prepared by Oshinei. This letter refers to the Oshinei document "API Specification for the Easy POS – Buzzebees Integration, Draft 1.1" (English and Thai editions), which remains the reference for test cases and business rules.